Preparation and Formatting of a Final Thesis

Peter Klepec

Master´s study programme Computer and Information Science, study field Computer and Information Science

Faculty of Computer and Information Science

Master's thesis

Supervisor: Assoc. Prof. Dr. Jason File
Cosupervisor: Prof. Dr. Justin Byte
Cosupervisor: Prof. Dr. Example Second Cosupervisor

Copyright: This work is licensed under a CC-BY-4.0 license. The software and results produced in this work are released under GPL-3.0-or-later at https://git.fri.uni-lj.si/UL-FRI/thesis.

This final thesis explores the practical application and advantages of the LaTeX typesetting system used directly in the preparation of this specific work. Writing a comprehensive technical text at the Faculty of Computer and Information Science (FRI) presents numerous challenges regarding typography, formatting consistency, and bibliography management. Traditional visual word processors frequently introduce uneven spacing and alignment issues, whereas LaTeX utilizes a logical approach to text processing that ensures a high degree of document uniformity, consistency, and portability across this entire document. The thesis provides a detailed overview of the structure implemented in this final thesis, stylistic guidelines, and the correct integration and referencing of key elements, including floats (figures and tables), code snippets, and pseudocode. Special attention is dedicated to established Slovenian typographical rules and the proper notation of mathematical constructs and matrices. Furthermore, we demonstrate the integration of the BibLaTeX system for the automated, standardized management and citation of the referenced literature. The presented text serves as a direct practical guide and a technical template, enabling students to efficiently structure their research and achieve strict compliance with academic standards.

Final Thesis | LaTeX | Typography | Text Formatting | Technical Writing

Priprava in oblikovanje zaključnega dela

V tem zaključnem delu obravnavamo praktično uporabo in prednosti stavčnega sistema LaTeX pri pripravi tega konkretnega zaključnega dela. Pisanje obsežnejšega strokovnega besedila na Fakulteti za računalništvo in informatiko (FRI) prinaša številne izzive na področju tipografije, doslednosti oblikovanja in urejanja bibliografije. Tradicionalni vizualni urejevalniki besedil pogosto povzročajo neenakomerne razmike in težave pri poravnavi, medtem ko LaTeX z logičnim pristopom zagotavlja visoko stopnjo uniformnosti, konsistentnosti in prenosljivosti tega dokumenta. V delu podrobno predstavljamo strukturo, ki jo uporablja to zaključno delo, slogovne smernice ter pravilno vključevanje in sklicevanje na ključne elemente, kot so plovke (slike in tabele), programski izseki ter psevdokoda. Poseben poudarek namenimo uveljavljenim slovenskim tipografskim pravilom in pravilnemu zapisu matematičnih konstruktov in matrik. Prav tako demonstriramo integracijo sistema BibLaTeX za avtomatizirano in standardizirano upravljanje ter citiranje uporabljene literature. To besedilo služi kot neposreden praktični vodnik in tehnična predloga, ki študentom omogoča učinkovito strukturiranje lastnih misli ter doseganje stroge skladnosti in z zahtevanimi akademskimi standardi.

Zaključno delo | LaTeX | Tipografija | Oblikovanje besedil | Tehnično pisanje

The first useful piece of advice regarding the use of LaTeX when writing your final thesis is to read this entire document!

This file briefly describes how to approach writing a final thesis using the LaTeX software environment [1, 2].

LaTeX enables logical text processing, which offers numerous advantages compared to visual word processing, as it approaches the problem of text formatting from a programmer's perspective. Logical text formatting provides greater consistency, uniformity, and portability of documents, and above all, it enables automatic hyphenation and consequently clean full justification. Full justification in visual word processors often results in numerous excessive and uneven spaces.

Section 1 briefly presents the typical components of a final thesis and stylistic guidelines. In Section 2, we introduce the basic building blocks of LaTeX, writing tips, the use of quantities and units, and the rules for cross-referencing and citations. In Section 3, we highlight the most common grammatical and typographical mistakes made when writing in Slovene (as well as general punctuation and spacing rules). Section 4 presents the inclusion of floats: inserting and formatting individual figures and tables, as well as placing multiple elements side by side. In Section 5, we learn how to include source code and algorithms. In Section 6, mathematical environments, the notation of matrices and systems of equations, text constructs (such as theorems and proofs), and referencing them are presented. In Section 7, we cover finding sources and citing literature using BibLaTeX. Finally, concluding remarks follow with a checklist for verification before submitting the thesis.

This template is not a comprehensive manual for using LaTeX, as it explains only some basic commands and merely mentions other features. The reader is encouraged to look elsewhere for details on how to use them.

1. Structure of the Final Thesis

While a final thesis has a foreseen general outline, the actual structure depends heavily on the field in which you work, so be sure to consult your supervisor regarding this. A well-established structure enables the reader—and above all the supervisor and defense committee members—to navigate the text quickly and verify the scientific integrity of the completed work.

A typical structure of a final thesis at FRI comprises mandatory and recommended components listed below. The expected length of final theses in this template, including references but excluding any appendices, is:

- 8 to 12 pages for Bachelor's theses and
- 11 to 20 pages for Master's theses.

The following applies to both:

- the recommended length of the abstract in both Slovene and English is 150 to 250 words,

- the recommended length of the extended Slovene abstract (at the end of the thesis, before any appendices) is 10 % of the length of the entire work (excluding appendices),
- 4 to 6 keywords are provided.

General components of a final thesis:

Title page in addition to the title in Slovene and English, contains the name of the author, supervisor, potential cosupervisor, and the year and place of publication. The title should be concise yet informative enough that the reader immediately recognizes the computer science domain addressed by the thesis.

Abstract (Slovene and English) is a short, self-contained text (usually limited to approximately 250 words) that summarizes the motivation, methods used, and most important results. At FRI, both a Slovene abstract (*Povzetek*) and an English translation (*Abstract*) are mandatory, increasing the visibility of the work in the international community.

Keywords are selected terms used for indexing the thesis in digital libraries. You should choose words that specifically describe the technologies, algorithms, or domains (e.g., machine learning, React framework, generative models).

Introduction sets the stage for understanding the problem. In it, we clearly define the state of the art, motivation for the work, thesis objectives, and hypotheses we wish to verify. The introduction usually concludes with a brief overview of the contents by individual sections, serving as a roadmap for the reader. Be careful not to write section numbers manually; instead, reference them with `\ref{xx}` and make sure to include `\label{xx}` at the section headings.

Background and Related Work is intended to place the thesis within the existing context. Here, you present which solutions for the chosen problem already exist and how your solution differs from those developed by other authors. A thorough literature review demonstrates your familiarity with the field.

Core Sections form the backbone of the thesis. In computer science theses, they typically cover the following sections, though you may split them into multiple separate sections and subsections, name them differently, or omit them, depending entirely on your field. Consult your supervisor primarily regarding this:

- **Methodology or Theoretical Background:** Explanation of concepts required to understand the solution.
- **Analysis and Design:** Description of the solution architecture, constraints, requirements, etc.
- **Implementation:** Description of the solution, development details, libraries used, and resolution of specific technical challenges, but without actual experimental numbers.
- **Experiments, Evaluation, and Results:** Testing the solution, performance comparison or user study, presentation of results, and discussion. Typically, this is split into multiple sections depending on the number of experiments conducted.

Conclusion summarizes the achieved objectives and provides a critical assessment of the completed work. It is important to highlight the limitations of your solution and outline directions for potential future work or improvements.

Bibliography is a list of all professional and scientific articles, books, and web sources that you reference. Every source in the list must have a corresponding citation in the text.

Appendices are intended for material that would disrupt the flow of the main text too much, yet is important for complete understanding (e.g., extensive code listings, questionnaires, or detailed diagrams).

1.1. Stylistic Guidelines and Recommendations. Keep in mind that the order of reading does not follow the order of writing a final thesis. You will likely tackle writing the related work and methodology before the experiments or results, and certainly before the introduction or abstract, which you should definitely write at the very end. Readers are aware of this as well, so when writing the introduction, you do not need to pretend that the experiments have not yet been conducted.

- Technical texts must be unambiguous, so sentences should be short, simple, and clear, and terminology must be consistent.
- When writing your final thesis, follow an academic style, which differs significantly from popular writing. Write the text in the first-person plural (e.g., "we developed", "we observe"), as this emphasizes the objectivity of the research process involving both the student and the supervisor. The passive voice is also permitted, although it can be more cumbersome ("it was developed", "it was designed"). Avoid using the first-person singular ("I developed", "I designed"), or use it only exceptionally where you wish to emphasize your personal choice among several options.
- All claims in the thesis must be verifiable. This means they must be based on your own derivations or tests; if you summarize the findings of others, citing the source is mandatory.
- Use the same name and symbol (if needed) for each individual concept or object throughout the entire work.
- If you introduce new concepts, make sure they are unambiguously and precisely defined.
- Use established technical terminology.
- Writing technical texts in Slovene also contributes to the development of Slovene technical terminology. For technical terms that do not yet have an established translation in Slovene, find an appropriate Slovene translation as needed in cooperation with your supervisor.
- When writing in Slovene (including the Slovene abstract and extended abstract), pay special attention to translations from English and follow Slovene grammatical rules. Noun adjuncts (uninflected modifiers/acronyms) in Slovene belong to the right of the head noun (SQL query → *poizvedba SQL*, API interface → *vmesnik API*, x-axis → *os* x); the reverse order, such as *SQL poizvedba*, *Windows strežnik*, or *ACME sistem*, is incorrect in Slovene.

2. Basic Building Blocks of LaTeX and Writing Tips

LaTeX can be most simply described as a programming language designed for typesetting text. Like any high-level programming language, LaTeX has numerous commands for formatting text and environments that enable structuring the document.

All LaTeX commands start with a backslash `\\`, whereas environments are defined either with a pair of curly braces { and } or with the commands `\begin{ }` and `\end{ }`. Commands can also take arguments: mandatory arguments are enclosed in curly braces, while optional arguments are enclosed in square brackets.

Using commands, we define the title and author names of the document, sections and subsections, and, as needed, structure the text in greater detail into lists, quotations, etc. Special environments are dedicated to typesetting mathematical expressions; short examples are provided in a later section.

All text constructs can be labeled, and using these labels, we can reference them anywhere in the text.

LaTeX automatically arranges words into paragraphs by optimizing the spacing between words across the entire paragraph. We start a new paragraph by leaving a blank line in the source text. To force a line break without starting a new paragraph, we use two backslashes (`\\`). The number of spaces between words in the source text does not matter.

2.1. Useful Tips for Writing in LaTeX. The LaTeX software package was originally introduced in the manual [1] and is actually an extension of the TeX system created by Donald Knuth [3], known for his books on the art of computer programming and the Knuth–Bendix algorithm [4]. TeX and its derivatives are open-source software.

There are many different implementations of LaTeX, but we recommend the online version that simplifies collaborative writing, Overleaf.

In the past, when writing in LaTeX, we relied primarily on printed manuals [1], whereas today it is simpler and faster to simply ask your favorite large language model for help whenever a problem arises.

LaTeX sometimes cannot properly hyphenate words (especially Slovene words containing characters with carons). If such a word protrudes past the right margin, we can show LaTeX where the word may be hyphenated like this: `ra\-ču\-nal\-ni\-štvo`. We can see which lines are overfull by compiling the document with the `draft` option enabled: `\documentclass[a4paper, 12pt, draft]{book}`.

In the source file, you can start each sentence on a new line, since LaTeX automatically arranges words across the lines of the typeset text. This also makes searching and editing the source text much faster. Most TeX environments also allow you to easily jump from the compiled PDF to the corresponding location in the source text and vice versa by clicking (SyncTeX).

Better readability is achieved just as when writing source code—by visually formatting the code and leaving blank lines. When modifying and adding source text, it is best to compile frequently to verify right away whether our intentions are properly fulfilled.

When we wish to transfer text that has already been written in a visual word processor (e.g., Word) into LaTeX, it is also best to do so gradually in individual blocks of text, so that any errors can be quickly identified and fixed. Converters for translating Word files into LaTeX —and vice versa—do exist, but they usually do not generate as clean a logical text structure as LaTeX otherwise allows. A quick and easy way to transfer text, which still requires manual adjustments, is to first save the text from the visual word processor in PDF format, and then import or copy that text into the editor where we edit the LaTeX source file.

2.2. Fonts in LaTeX. In a LaTeX environment, we can in principle use arbitrary fonts. However, choosing an arbitrary font is not as straightforward as in visual word processors. Individual stylistically coordinated fonts are usually grouped into font families. In LaTeX, the default font family is called Computer Modern, where in addition to upright letters (roman in LaTeX), italic letters (*italic* in LaTeX), boldface (**bold** in LaTeX), small capitals (SMALL CAPS in LaTeX), sans-serif letters (sans serif in LaTeX), monospaced typewriter font (`typewriter` in LaTeX), and some of their combinations, e.g., bold sans-serif letters (**sans serif** in LaTeX), are available. For a consistent appearance within the same document, we usually use fonts from only a single font family. Consistent use of different font styles is also important, as is avoiding excessive mixing of different fonts.

When starting out with LaTeX, it is therefore most sensible to simply use the default fonts, in which this document is also written. With appropriate commands, we can then switch between upright, italic, bold, and other font styles. Selecting font sizes is also very straightforward. LaTeX provides excellent support for multilingual documents, even within the same document, as fonts exist for virtually all languages, including those that do not use the Latin script.

2.3. PDF/A Compliance. The Repository of the University of Ljubljana requires submitted final theses to comply with the PDF/A standard. In this template, we have ensured compliance with the PDF/A-2b standard. Unfortunately, the template cannot prevent a user from adding elements to the thesis that are not compliant with the standard. Before submitting the final thesis, we therefore recommend that users verify compliance using appropriate validators, e.g., the veraPDF tool[1].

2.4. Using Quantities and Units. In technical and scientific writing, it is very important to consistently distinguish between physical/technical quantities and their corresponding units of measurement. Improper notation can lead to misinterpretation of data and diminish the professional quality of the document.

Quantity represents a property of a system, hardware, or software that can be determined quantitatively. Quantity symbols (e.g., C for memory capacity, f for processor frequency, B for bandwidth) are written in **italic type** as a rule.

Unit of measurement is an agreed reference value with which we compare quantities of the same kind. Unit symbols (e.g., B, Hz, $\mathrm{bit\,s^{-1}}$) are written in **upright (roman) type**.

Between the numerical value and its corresponding unit, there must always be a **non-breaking space** (tilde), just as with citations. This ensures that the value and the unit remain together on the same line at a line break.

To achieve standardized notation in LaTeX, we use the `siunitx` package. It provides automatic management of spacing and decimal markers. Examples of proper usage are given below:

[1] https://verapdf.org/

- **Printing a value with a unit:** The command `\qty{100}{\giga\byte}` produces 100 GB.
- **Special symbols:** Percentages and degrees are written with a space in Slovene and handled automatically by `siunitx`, e.g.:
  - `\qty{98}{\percent}` produces 98 %,
  - `\qty{85}{\degreeCelsius}` produces 85 °C.

In English, a period is used as the decimal marker (the default in `siunitx`), whereas in Slovene a comma is used, which can be set globally in `siunitx` with `output-decimal-marker = {,}`.

Here is another example of using units in text: During the server stress test, we found that at a frequency of $f = 3.5\,\mathrm{GHz}$ and a load of 98 %, the network throughput reaches $B = 1.2\,\mathrm{Gbit\,s^{-1}}$. All measurements were performed at a processor temperature of $T = 85\,^{\circ}\mathrm{C} \pm 1.5\,^{\circ}\mathrm{C}$.

2.4.1. Computer Scientists, Respect the SI. When writing, keep in mind that the entire world, with the exception of Liberia, Myanmar, and the United States of America, officially uses the International System of Units (SI), which makes our everyday lives easier. Isn't it wonderful to say that you drove 23 Mm this year, instead of the uninitiated rambling about 23 000 km?

When writing sizes in bytes, frequent issues arise for historical reasons. The following rules apply:

- If you used a decimal multiplier (1000), use SI prefixes: k (kilo), M (mega), G (giga), T (tera), i.e., kB, MB, GB, TB.
- If you used a binary multiplier (1024), use binary prefixes: kibi (Ki), mebi (Mi), gibi (Gi), tebi (Ti), i.e., KiB, MiB, GiB, TiB. Do not be distracted by the fact that many manufacturers and even certain operating systems fail to follow this standard (perhaps they come from one of the three special countries).

Of course, do not forget the difference in notation between a byte (B) and a bit (bit)!

2.5. Cross-References and Citations. When writing technical texts, it is essential to consistently reference figures, tables, equations, and other sections, as well as to cite the literature used. The LaTeX system makes this much easier, as it automatically handles proper numbering during compilation. We use the following basic commands:

- **Labeling (`\label`):** An element that we want to reference must first be marked with the command `\label{label_name}`. For better organization, it is recommended to use prefixes, for example `fig:` for figures, `tab:` for tables, and `sec:` for sections (e.g., `\label{sec:osnove}`).
- **Referencing (`\ref`):** In the text, we reference a labeled element using the command `~\ref{label_name}`. LaTeX will print the corresponding number at that spot. When referencing a mathematical equation, we use the command `~\eqref{label_name}` instead, which automatically encloses the equation number in parentheses.
- **Citing (`\cite`):** To cite sources from a bibliography file, we use the command `~\cite{source_key}`.

A very important rule: Between the preceding word and the reference or citation itself, there must always be a **non-breaking space**, denoted in LaTeX by a tilde (`~`). This prevents the reference number from falling alone onto a new line at a line break, which is typographically incorrect.

Proper usage in the text therefore looks like this: As we can see in Figure`~\ref{sl:example_coral_reef}`, Equation`~\eqref{en:masa}` holds. Similarly for literature: The algorithm is described in Section`~\cite{knuth}`.

To reference unnumbered chapters and subsections by title, we can use the `\nameref` command, which prints the title of the section or subsection, e.g., in Section Floats: Figures and Tables or Subsection Image Formats.

2.6. Lists. The template supports several environments that create lists. We divide them into regular (`itemize` and `enumerate`) and compact environments (`compactitemize` and `compactenumerate`). Regular unordered and ordered lists are suitable when we want to list several items with longer descriptions. We have already seen long descriptions in unordered lists in some of the earlier examples, so we will not repeat them here. Let us demonstrate only the use of compact unordered and ordered lists. Note that optional arguments are also supported, just as in standard `itemize` and `enumerate` environments.

Here is an example of an unordered list:

- first condition,
- second condition,
- third condition,

and an example of an ordered list:

a) first prerequisite,
b) second prerequisite.

3. Common Mistakes When Writing in Slovene (and General Typography)

When writing technical texts in Slovene (including the mandatory Slovene abstract and extended Slovene abstract), authors frequently encounter typical errors stemming from the influence of foreign languages or unfamiliarity with LaTeX typographical rules. Below are the key rules, organized by category:

- **Word and acronym order in Slovene:** Adjectives/modifiers that do not inflect (e.g., acronyms) must stand after the noun in Slovene. In Slovene, we correctly write "model CAD" and **not** "CAD model".

- **Inflecting foreign names in Slovene:** When inflecting foreign proper names in Slovene, we do not use hyphens. In Slovene, the correct form is "Applov operacijski sistem" and **not** "Apple-ov".

- **Punctuation and spacing:** Periods, exclamation marks, and question marks are attached to the preceding word (no space before them, followed by a space). Exclamation marks and question marks are generally avoided in academic texts. Opening parentheses are attached to the following word, and closing parentheses to the preceding word: "(like this)".

- **Ellipses and enumeration:** Simply do not use ellipses (. . .) in final theses. In scientific texts and final theses, use "etc." (or "itd."/"idr." in Slovene) rather than an ellipsis. Be careful to use only "e.g." (or "for example") or "etc." in a single clause, never both together.

  For using ellipses elsewhere in life, the following applies: in Slovene, an ellipsis is generally preceded by a space. In LaTeX, we write it using the command `\dots` and not with three periods. Example: "dog, cat, wolverine, . . . " An exception in Slovene is an ellipsis followed by another punctuation mark, where there is no space before the second mark, e.g., "dog, cat, wolverine, . . . !"

- **Quotation marks:** Quotation marks attach directly to the quoted word without inner spaces. In LaTeX, we must not simply use the `"` key or two apostrophes `''` for both sides, as opening and closing quotation marks will not be rendered properly. Best practice is to use the command `\enquote{...}`, which automatically places the correct quotation marks for the active document language: "quoted text" (or in Slovene: lower nines and upper sixes).

- **Hyphens and dashes:**

  - **Hyphen (-):** Attached on both left and right, e.g., `Slovene-English` (`slovensko-angleški`), indicating a compound or symmetric relationship between concepts. In source code, we write it with a single character `-`.

  - **En-dash as a parenthetical dash (–):** In Slovene and in British English, it is surrounded by spaces. In source code, we write it with two characters: `\text{Beware -- vicious dog!}` (`\text{Pozor -- hud pes!}`).

  - **En-dash for ranges (–):** Used for ranges (e.g., "from–to"), written without spaces: `pages 7--11` (`strani 7--11`), and typed in code in the exact same way as an en-dash (`--`).

  - **American em-dash (—):** The *em-dash* is longer, written without spaces, and typed in code with three characters `---`. It serves the exact same parenthetical role in American English as the spaced en-dash in Slovene or British English, e.g., `Beware---vicious dog!` Do not use it when writing in Slovene (and be consistent with either British or American dash style in English).

- **Spacing around punctuation and abbreviations:** By default, LaTeX uses non-uniform spacing (traditional English typesetting spacing, often contrasted with French spacing), which means the space after the end of a sentence is slightly larger than the space between words. This leads to two specific issues:

  - **Abbreviations in the middle of a sentence:** If a period after an abbreviation (e.g., *Prof.*, *Dr.*, *e.g.*, *etc.*) is followed by a regular space, LaTeX will interpret it as the end of a sentence and insert an overly wide space. We prevent this by using a tilde `~` (non-breaking space) or a backslash followed by a space. Correct: `Prof.~Dr.~Novak` or `e.g.\ in this case`.

  - **Acronyms at the end of a sentence:** If a sentence ends with an uppercase letter (e.g., an acronym at the end of a sentence), LaTeX assumes it is an initial in the middle of a sentence and inserts too small a space. For proper end-of-sentence spacing, insert the command `\@` before the period.
    Correct: `They developed a new CPU\@. The testing phase followed.`

- **Commas and sentence structure (in Slovene):** In Slovene, a comma separates clauses (predicates), unless they are joined by coordinating conjunctions *in, pa, ter, ne–ne, niti–niti, ali, bodisi, oziroma.* Although the rule of thumb states that subordinate clauses introduced by *ki, ko, ker, da, če* are preceded by a comma, one must watch out for parenthetical clauses and multi-word conjunctions. In technical texts, it is better to write shorter, clear sentences. Large language models can be of great help with comma placement, and for truly accurate comma placement in Slovene, we recommend using the online tool CJVT Vejice (https://orodja.cjvt.si/vejice/).

- **Terminology and resources (for Slovene):** For new computer science terms in Slovene, consult established dictionaries. The main resources are *Islovar* by the Slovene Society Informatika [5], the Pattern Recognition Dictionary by the Slovene Pattern Recognition Society [6], and the central linguistic portal *Fran* [7].

4. Floats: Figures and Tables

Figures and longer tables are generally included in the document as floats. The position of a float in the final document is not determined strictly by the flow of the text, but rather by page layout considerations. LaTeX will try to place the float independently, ideally at the spot where it appears in the source text, or otherwise at the top of the page on which such a float is first referenced. In doing so, it also aims to keep a reasonably large portion of text on every page of the final document. In extreme cases, if we have too many floats in one part of the text, or if a float is too tall, LaTeX will opt for a dedicated page filled entirely with floats.

Besides influencing the position of a float by where we place it in the source text, we can further influence its placement on an individual page of the compiled document using the options `here, top` (`h, t`) and `bottom` (`b`), though you should avoid restricting this if at all possible. Very large figures are best placed on a dedicated page using the `page` (`p`) option. We can scale the width of figures relative to the page/text width by using the text width itself as the unit, e.g., `0.5\textwidth` will scale the figure to half the width of the text block. If needed, a figure can also be rotated by 90 degrees and stretched to the height of the page. This makes details in the figure easier to read and utilizes the space on the page better.

All floats must be referenced in the text because, as the word float implies, floats move within the text and do not necessarily appear at the exact spot where they are placed in the source file. However, the reference to a float in the text and the float itself in the typeset document should be as close to each other as possible, so that the reader does not have to flip pages back and forth. Keep in mind that theses are formatted for two-sided viewing/printing, meaning two facing pages are visible at once! The most effective way to influence where a figure or other float appears in the typeset text is therefore to move the float backward or forward in the source code!

Tables are best formatted directly in LaTeX, as the very flexible `tabular` environment exists for creating tables. Figures, on the other hand, are often easiest to design or

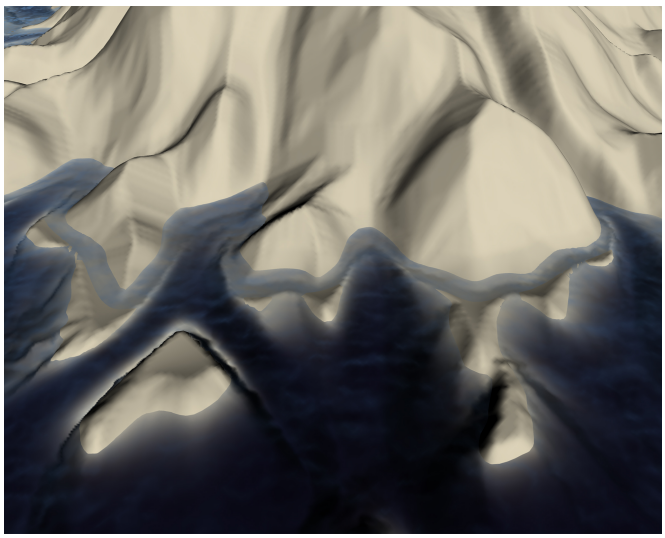


Figure 1. Virtually enhanced sculpture [8]. The results of a computer-generated animation are projected onto a stone sculpture using a video projector to create the impression of water droplets trickling down the sculpture [9, 10].

produce using other tools and programs, saving the results in `.pdf` or `.jpeg` format, and then simply including the appropriate image file in LaTeX. For text included inside figures, use fonts that are as similar as possible to the fonts in the main text, or even better, export without embedded font outlines/use LaTeX rendering so LaTeX can use its own fonts.

The library https://en.wikibooks.org/wiki/LaTeX/PGF/TikZ also enables drawing a wide variety of diagrams and plots directly within the LaTeX environment.

Every table and figure must be referenced in the text, since as floats in the typeset document they are not necessarily at the same position as in the source text. When referencing specific numbered figures or tables in the text, capitalize the noun, e.g., "see Figure 1", as it acts as a proper identifier.

4.1. Image Formats. We can include images of various formats in a LaTeX document, both raster images and vector graphics.

Vector graphics in `.pdf` format are most suitable for plots and diagrams. Not only will the text inside them be represented as actual text, but the figures will remain crisp and legible even at high magnification. Moreover, saving in vector format usually requires no extra effort on your part. For example, if you plot a chart in Python, simply make sure to save it as `.pdf` rather than `.png`, and certainly never as `.jpg`.

If you are including photographs or something you did not draw yourself, it will likely be stored in raster format and saved as `.jpg` or `.png`. Figure 1, for example, is in `.jpg` format, but you include a `.pdf` file in the exact same way.

4.2. Captions for Figures and Tables. Every figure or table must be accompanied by a caption that explains what is shown in the figure or table, defines any abbreviations used, and summarizes the main takeaways. Do not be afraid of longer descriptions. If someone merely skims through the final thesis, they should be able to get a rough idea of what the thesis is about just from the figures and their captions.

If figures are adapted or taken from other sources, it is mandatory to cite the source in the caption of such a figure!

4.3. Including Multiple Figures and Tables Side by Side. When writing a final thesis, there is often a need to display multiple figures or tables side by side. For this purpose, we use the `subcaption` package, which represents the modern and supported standard in LaTeX.

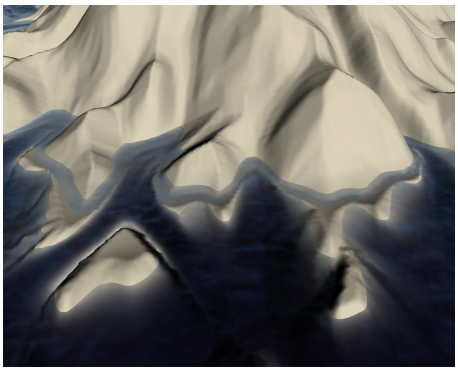
In online resources and old templates, students all too often find and use the `subfigure` or `subfig` packages. Both packages are obsolete and frequently cause compilation issues or conflicts with other packages (e.g., when generating hyperlinks). Therefore, **do not use** those two packages. Likewise, do not use the `TikZ` package just to arrange images into a grid. While `TikZ` is an extraordinarily powerful tool, it is intended for drawing vector diagrams and is far too complex for simply arranging images.

To use the modern approach, add the following command to the document preamble: `\usepackage{subcaption}`. The package provides two environments: `subfigure` for figures and `subtable` for tables. Behind the scenes, both environments work on a similar principle to the standard `minipage` environment (which is commonly used in LaTeX to

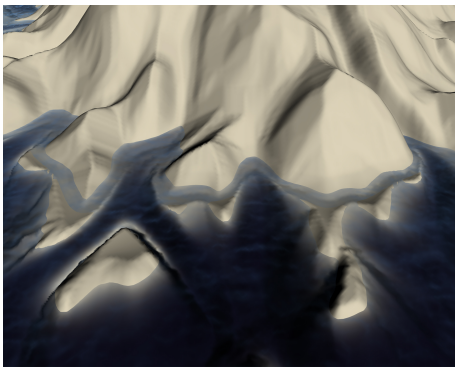
place completely independent blocks of content side by side), as we pass an explicit width to each of them.

Because figures and tables are floating objects, we must never assume where exactly on the page they will be rendered. Always reference all elements in the text exclusively using the `\ref` command preceded by a non-breaking space (`~`).

4.3.1. Multiple Figures with Subcaptions. When we want to assign each figure its own label (e.g., a, b) and subcaption, we use the `subfigure` environment. We specify the width of each subfigure as a fraction of the text width, e.g., `0.48\textwidth`. We fill the remaining horizontal space up to the full width with the command `\hfill`. In Figure 2, we can see an example of two figures placed side by side.



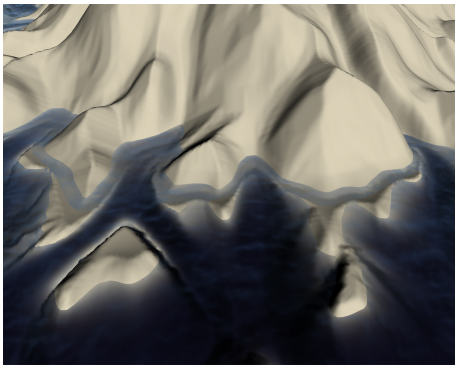
(a) Subcaption of the left figure.



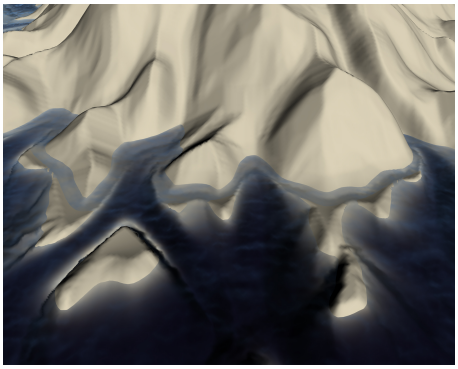
(b) Subcaption of the right figure.

Figure 2. Common caption describing both figures.

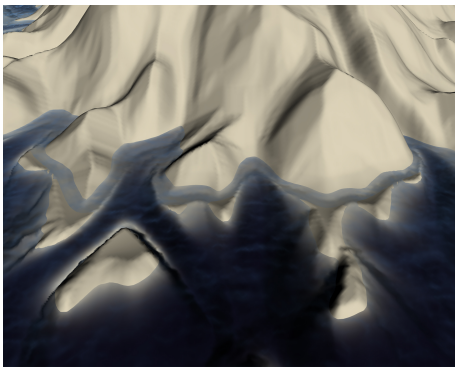
To move to a new row within the same floating object, we simply leave a blank line and optionally add some vertical space with the `\vspace` command. A grid of four figures in Figure 3 is constructed in this way.



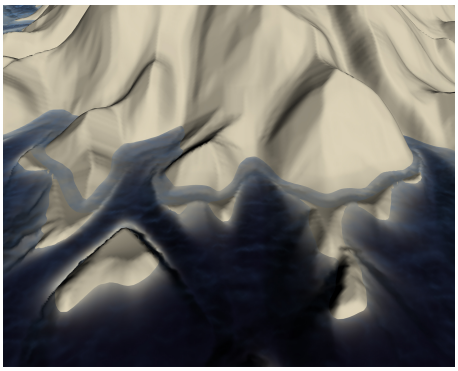
(a) Top left.



(b) Top right.



(c) Bottom left.



(d) Bottom right.

Figure 3. A grid of four figures with individual subcaptions.

4.3.2. Multiple Figures Without Subcaptions (Zero-Gap Grid). If individual subcaptions are not needed and we only want a single common caption for the entire image grid, we do not

need the `subfigure` environment at all. We simply stack the images inside a regular `figure` environment and specify their sizes.

If we want to create a continuous grid completely **without intermediate gaps** (as shown in Figure 4), we must take care of two things:

1. **Horizontal spaces:** Specify the width so that the sum of all widths in a row equals exactly `1.0\textwidth` (or `1.0\linewidth`, e.g., four images with width `0.25\linewidth`). At the end of every line with an `\includegraphics` command, we must add a comment character (`%`) to prevent LaTeX from interpreting the line break in the source code as a space.

2. **Vertical spaces:** Inside a dedicated group, we must use the command `\offinterlineskip`, which completely disables automatic insertion of vertical space between lines. It is important to close the group with } after the last image before defining `\caption`.

In Figure 4, a wider layout is also used by employing the `{largefigure}` environment instead of `{figure}`.

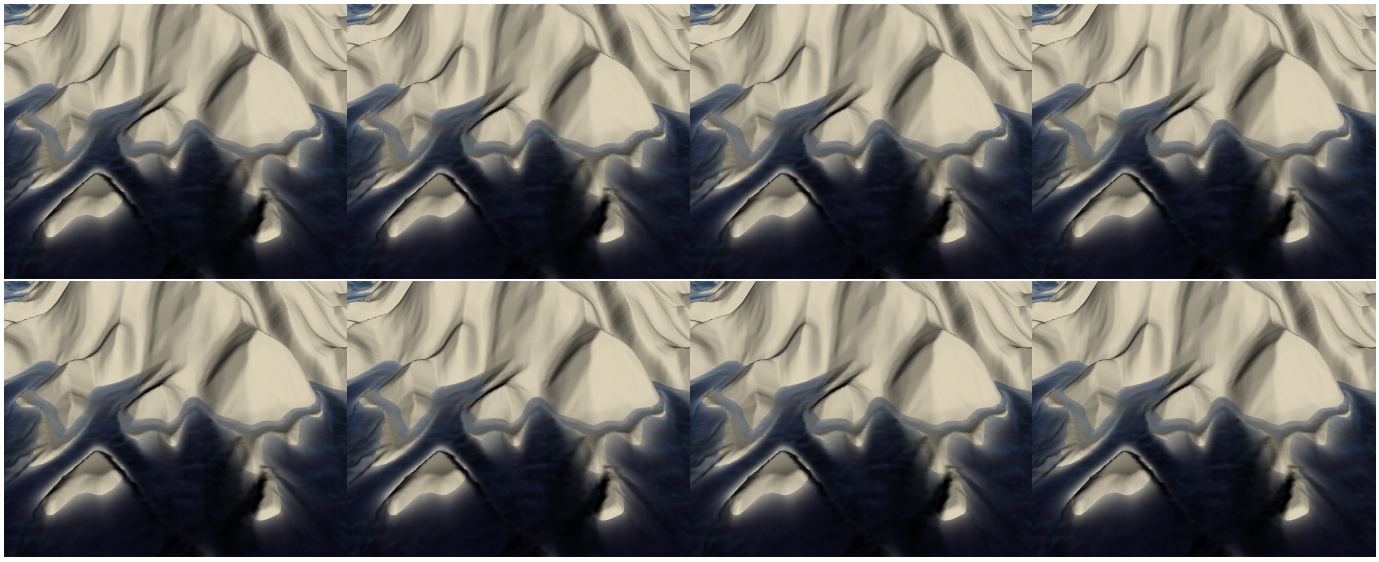


Figure 4. A grid of eight images without intermediate gaps and with only a single common caption. A wider layout is also demonstrated.

4.3.3. Multiple Tables Side by Side. The logic of placing two tables side by side is completely identical to that for figures, except that inside the `table` environment we use the `subtable` environment. The only important rule you must not forget is: **table captions always belong above the table**, whereas figure captions belong below the figure.

If you find that the content of a table is too wide and causes overlapping, you can wrap the entire `tabular` environment in a `\resizebox{\linewidth}{!}{...}` command, which scales the table down proportionally so that it fits the width of the subtable exactly. Table 1 demonstrates proper notation on an example of evaluating computer vision models.

Table 1. Comparison of image classification performance across different deep neural network architectures on two standard datasets. For the *accuracy* metric, higher values are better (indicated by ↑), whereas for the *error rate* metric, lower values are better (indicated by ↓).

(a) Results on the CIFAR-10 dataset.

architecture	accuracy (↑)	error rate (↓)
ResNet-18	93.5 %	6.5 %
ResNet-50	95.2 %	4.8 %
ViT-Base	97.8 %	2.2 %

(b) Results on the ImageNet dataset.

architecture	accuracy (↑)	error rate (↓)
ResNet-18	69.7 %	30.3 %
ResNet-50	76.1 %	23.9 %
ViT-Base	84.2 %	15.8 %

The use of wide tables is also supported, as seen in Table 2.

5. Including Source Code and Algorithms

The easiest way to include your source code in any programming language is with the `minted` package. While you can wrap a `\minted{ }` block inside a standard `\listing{ }` float and add a `\caption` to get everything needed to display code, such a floating block

Table 2. Example of a wide table. Execution times of three fictional algorithms are shown. It is evident that Algorithm A has exponential time complexity, Algorithm B has quadratic time complexity, and Algorithm C has linear time complexity. While such data is generally better presented on a plot, this should suffice to demonstrate the use of a wide table.

n	execution time of Algorithm A	execution time of Algorithm B	execution time of Algorithm C
5	2 s	3 s	4 s
10	61 s	9 s	6 s
15	33 min	18 s	8 s
20	17 h	32 s	10 s

does not break properly across multiple pages. One possible solution is to use a `\center{ }` block and `\captionof{ }` for the caption, as shown in both code listing examples 1 and 2.

Listing 1. A short example of code in the Python programming language.

```python
def python_squirrel():
    # Oh, how nice this is!
    x = -1
    print("It really looks wonderful.")
```

What is especially great about the `minted` package is that we can also include inline code snippets within a line of text, such as this: `python_squirrel()`.

Listing 2. A slightly longer example of source code in JavaScript that can span across two pages.

```javascript
async function fetchUser(id) {
    const url = `https://api.example.fri/users/${id}`;

    try {
        const response = await fetch(url);

        // Check if the status code is in the 200-299 range
        if (!response.ok) {
            console.error(`Connection error: ${response.status}`);
            throw new Error(`User with ID ${id} could not be found.`);
        }

        const data = await response.json();

        // Log successful data retrieval
        console.log("Data received successfully.");
        console.log(`User name: ${data.name}`);

        return data;

    } catch (error) {
        // Catch both network errors and JSON parsing errors here
        console.warn("An unexpected issue occurred.");
        return { error: error.message };
    }
}
```

The horizontal lines at the beginning and end of the snippet, combined with the gray background, visually indicate when a listing spans across multiple pages. This can be seen in Listing 2.

5.1. Including Algorithms (Pseudocode). While we use the `minted` package to display an actual implementation in a specific programming language, in final theses we often describe only the problem-solving logic at a higher level. For writing algorithms in the form of pseudocode, the standard choice is the combination of the `algorithm` and `algpseudocode` packages (the latter is part of the `algorithmicx` bundle).

Unlike source code snippets, algorithms are usually treated as standard floating objects (just like figures and tables). This means that LaTeX automatically finds the optimal place to display them on the page. In the text, we reference them using a non-breaking space and the `\ref` command, for example: Algorithm 1 shows the classic procedure for finding the greatest common divisor.

The caption (`\caption`) for algorithms is typically placed at the top, right below the start of the `algorithm` environment. The argument `[1]` at the beginning of the `algorithmic` block instructs LaTeX to number every line of the algorithm, making it easy to reference individual steps in the text.

Algorithm 1. Euclid's algorithm for finding the greatest common divisor

Require: $a \geq 0 \vee b \geq 0$
Ensure: Greatest common divisor of a and b

```
1: procedure EUCLID(a, b)
2:     while b ≠ 0 do
3:         t ← b
4:         b ← a mod b
5:         a ← t
6:     return a
```

6. Mathematical Environments and Referencing Text Constructs

Let us look at a few examples of writing equations, referencing them, writing proofs, matrices, splitting equations, and some general typographical rules in mathematics.

Theorem 1. *For every natural number n, it holds that*

$$n < 2^n. \tag{1}$$

Proof. Proof by induction requires that we first verify inequality (1) for the smallest natural number – 0. Indeed, since $0 < 1 = 2^0$, inequality (1) holds for $n = 0$.

We referenced Equation (1) using `~\eqref{eq:1}` rather than `~\ref{eq:1}`, as the latter does not enclose the reference in parentheses. Without parentheses, the reader might confuse an equation reference with a section reference, and with square brackets, with a literature citation.

For the sake of our example, let us continue with the induction step. Assuming that inequality (1) holds for some natural number n, we need to show that the same inequality also holds for its successor – the natural number $n + 1$. Let us calculate:

$$\begin{aligned} n + 1 &< 2^n + 1 && (2)\\ &\leq 2^n + 2^n && (3)\\ &= 2^{n+1}. \end{aligned}$$

Inequality (2) follows from the induction hypothesis, and inequality (3) from the simple fact that for every natural number n, the expression 2^n is at least as large as 1. This concludes the proof of Theorem 1. □

Did you notice how nicely aligned the induction steps are? We used the important `align` environment. Did you notice that we ended the sentence at inequality (1) with a period? When an equation is part of a sentence that continues afterwards, such as

$$y = 2^x, \tag{4}$$

we of course place a comma inside the equation itself, as shown in (4).

For high-quality technical writing, we also need to master writing matrices and handling very long equations.

6.1. Matrices.

To write matrices, we use environments from the `amsmath` package. Most commonly, we use the `bmatrix` environment for matrices with square brackets or `pmatrix` for matrices with parentheses.

Example of writing a matrix A:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}. \tag{5}$$

Here, we separate elements within a row using the `\&` character and end rows with `\\`. For different types of ellipses, we use the commands `\dots` (horizontal), `\vdots` (vertical), and `\ddots` (diagonal).

6.2. Splitting Long Equations.

When an equation is too long to fit on a single line, we use the `split` environment inside the `equation` environment. This allows us to break the equation across several lines while the entire block retains a single shared equation number.

$$\begin{aligned} f(x) &= (a + b)^4 \\ &= a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4. \end{aligned} \tag{6}$$

The `\&` character specifies the alignment point (usually at the equals sign), and `\\` marks a line break.

6.3. Systems of Equations and Piecewise Functions. To write piecewise-defined functions or systems of equations with a single large brace, we use the `cases` environment:

$$|x| = \begin{cases} x, & \text{if } x \geq 0, \\ -x, & \text{if } x < 0. \end{cases} \tag{7}$$

Notice the use of the `\text{...}` command, which allows inserting regular text (with commas and spaces) directly inside a mathematical environment while preserving the proper upright font.

6.4. General Mathematical Typography Rules and Environments. When writing mathematics, we follow these rules:

- **Commas and periods:** Equations are part of sentences and must therefore include appropriate punctuation. If a sentence ends with an equation, a period is placed at the end of the equation.
- **Standard functions:** Functions such as $\sin, \cos, \ln, \max$ are written in upright type using the corresponding commands (e.g., `\sin`), and not as sin (in italic type), which would denote the product of variables s, i, and n.
- **Spacing:** LaTeX automatically manages spacing in mathematics. Manual intervention is usually unnecessary, except in rare cases where we use, e.g., `\,` for a thin space.

The following environments for mathematical statements are also automatically translated according to the selected document language:

Definition 2. A problem Π is in the class NP if ...

Axiom 1 (Commutativity of Boolean Algebra)**.** Let $A, \vee, \wedge$ be a Boolean algebra. For every pair $x, y \in A$, it holds that:

$$x \vee y = y \vee x$$

and

$$x \wedge y = y \wedge x.$$

Proposition 3. *Let Σ be an NP-complete problem. If there exists a Karp reduction from problem Σ to problem Π, then Π is NP-hard.*

Remark 4. The existence of a Karp reduction from problem Π to problem Σ does not guarantee the NP-hardness of problem Π.

Theorem 5. *Suppose that*

Proof. ... □

Corollary 6. ...

Lemma 7. ...

Example 8. ... ♢

7. What About the Bibliography?

As mentioned in the introduction, the proper way to cite literature is by using BibLaTeX (or BibTeX) [11]. BibLaTeX ensures that for a given type of reference we do not omit any mandatory information and that all information is consistently formatted in the same way and in the same order. BibLaTeX is an upgrade to the older BibTeX system. The newer system is better adapted to Slovene and to citing web sources, while older `.bib` files remain compatible with BibLaTeX.

The basic idea of BibLaTeX is that all information about the literature is stored in a dedicated file, in our case `literatura.bib` (or `bib/navodila-za-pisanje.bib`). We assign a symbolic key to each source in this file. In our example, this file contains some of the most typical types of literature, such as books [1], journal articles [12] and conference proceedings papers [13], book chapters [14], web sources [9, 15], technical reports [16], theses [17], etc. The thesis [17] from 1990 was the first Bachelor's thesis at the then Faculty of Electrical Engineering and Computer Science to be typeset with LaTeX. References archived electronically online usually have a DOI identifier (http://dx.doi.org), which we therefore include in the bibliography entry to offer readers of the electronic version of our publication a direct link to the electronic copy of that reference.

After any change to a literature citation, we must first compile the source text with the LaTeX compiler, then with the BibLaTeX/BibTeX tool, which generates the `.bbl`

file, and then twice more with the LaTeX compiler. In the Overleaf environment, this multi-step compilation process is hidden from the user. Therefore, we also recommend Overleaf to beginner LaTeX users.

How the bibliography list is formatted (whether individual sources are ordered by appearance in the text or alphabetically by the first author's surname, whether first names are written before surnames, etc.) depends on the bibliography style parameters. With BibLaTeX, using the parameters `style=numeric` means that literature citations in the text are marked with sequential numbers, and for the sorting order of references, `sorting=nty` means that references are sorted by the first author's surname, then by title, and finally by publication year [18]. Therefore, for certain types of references that do not have authors, it is necessary to add the `key` field to specify the alphabetical sorting key of the source.

When starting out with BibLaTeX, it is easiest to use the same text editor for editing the `.bib` file as for editing `.tex` files, although dedicated reference managers and editors for working with `.bib` files also exist.

Only if a given source is actually cited in the text will it appear in the bibliography list. This automatically guarantees that every source listed in the bibliography is also referenced in the text of the final thesis. In the `.bib` file itself, we may keep many more references than we end up citing in the thesis.

7.1. Collecting Sources for the Bibliography. References in `.bib` format can easily be found and copied from publisher websites or various academic search portals for scientific literature. Exporting references from Google Scholar can be simplified even further by selecting BibTeX as the preferred citation export format in the settings. Citations copied from Google Scholar and similar academic portals must always be verified before use, as such entries are often generated completely automatically and may contain inaccurate or incomplete data. Most frequently, the publication type is wrong! And with large language models, an entire reference may be completely fabricated (hallucinated)!

When citing literature at the end of a sentence, make sure that the period comes after the `\cite{ }` command. To prevent LaTeX from breaking a line right before the citation so that the bracketed reference starts a new line on its own, we must place a non-breaking space (tilde) before the citation command: `~\cite{ }` and not a regular space!

Usually, we cite one or more sources at the end of a declarative sentence. However, when we mention the author of a source by name in the sentence, we usually place the citation immediately after their surname.

Nowadays, almost everyone starts searching for information via web search engines or large language models. However, the results of such searches are often web pages that exist today but may be gone tomorrow, or at least no longer in the form we read them. Yet the whole point of citing literature is that even many years later, someone reading your final thesis can locate the sources you cited.

Scientific results published as peer-reviewed articles—either in conference proceedings or, even better, in scientific journals—are a much more refined and reliable source of information, as such articles have undergone peer review. Above all, such articles are a stable source of information, since they generally do not change after publication. Almost all of these articles are nowadays available electronically, either in publisher archives, university repositories, or on the personal websites of their authors. Therefore, when searching online for sources for technical texts, we primarily use academic search portals such as Google Scholar, ResearchGate, or Academia.edu, as search results on these portals consist of academic publications. Be cautious with large language models, as references they suggest may be fabricated.

If access to an article requires a subscription or payment, contact our faculty library for assistance and access.

To identify articles available in electronic form, the DOI identifier (https://www.doi.org) has become standard, greatly facilitating locating these references online. Publishers also retroactively assign DOIs to older articles available electronically. Therefore, try to find the corresponding DOI for every article you cite and include it in your bibliography.

If there is truly no alternative to citing a regular web page, it is important to always include the date on which the web source was accessed (`urldate`).

Using BibLaTeX, it is also possible to print separate bibliography sections for specific reference types, for example journal articles, conference papers, and book chapters.

8. Concluding Remarks

The use of LaTeX and BibLaTeX (or BibTeX) is **mandatory** for the final thesis!

Before submitting your final thesis to your supervisor, double-check:

- proper use of elements (both LaTeX constructs and punctuation, abbreviations, dashes, etc.),

- grammar and spelling (you may use proofreading tools for assistance),
- that no text is pasted directly from a large language model without review (you must read, verify, and if necessary revise every single sentence yourself),
- that there are enough visual elements (excessive dry text is not helpful to anyone),
- that every figure and table is not only captioned, but also referenced in the text.

Before submitting your final thesis to the STUDIS system, double-check:

- whether the study programme, your name, and the name and title of your supervisor and cosupervisor are printed correctly,
- whether all words are properly hyphenated and do not protrude past the right margin (you can check line justification by compiling the source file once with the `draft` option, which marks overfull lines),
- whether you included the proper versions of figures, especially vector formats (`.pdf` or `.eps`) for plots and diagrams rather than raster formats (e.g., `.jpg` or `.png`),
- whether you have addressed all errors and warnings that may occur during document compilation (pay special attention to missing citations or broken references, which you can also spot by searching for question marks "?" in the final PDF).

ACKNOWLEDGEMENT. Here you may write whom you wish to thank for their help with your thesis or throughout your studies in general. Be careful not to forget anyone—they might hold a grudge. You can avoid this risk altogether by omitting the acknowledgements section entirely.

Bibliography

1. Lamport L (1986) *LaTEX: A Document Preparation System*. (Addison-Wesley).
2. Oetiker T, Partl H, Hyna I, Schlegl E (2006) Ne najkrajši uvod v LaTeX2ε.
3. Wikipedia contributors (2021) Donald knuth — Wikipedia, the free encyclopedia.
4. Knuth DE, Bendix PB (1983) Simple word problems in universal algebras in *Automation of Reasoning: Classical papers on computational logic 1957–1966*, eds. Siekmann JH, Wrightson G. (Springer), pp. 342–376.
5. Slovensko društvo Informatika (2021) Islovar.
6. Slovensko društvo za razpoznavanje vzorcev (2021) Slovar, Razpoznavanje vzorcev.
7. ZRC SAZU (2022) Slovarji Inštituta za slovenski jezik Frana Ramovša ZRC SAZU.
8. Solina F, Meden B (2017) Light fountain – a virtually enhanced stone sculpture. *Digital Creativity* 28(2):89–102.
9. Solina F (2015) Light fountain ii – galaxy.
10. Solina F (2021) *Skulpture/Sculptures 2012-2020, 2. izdaja / 2nd edition*. (Društvo likovnih umetnikov Ljubljana, Založba UL FRI, Ljubljana).
11. (1988) Bibliography management with BibLaTex.
12. Solina F (2004) 15 seconds of fame. *Leonardo* 37(2):105–110.
13. Ciuha P, Klemenc B, Solina F (2010) Visualization of concurrent tones in music with colours in *Proceedings of the 18th ACM international conference on Multimedia*. (ACM), pp. 1677–1680.
14. Peer P, Batagelj B (2009) Art—a perfect testbed for computer vision related research in *Recent Advances in Multimedia Signal Processing and Communications*. (Springer), pp. 611–629.
15. (year?) SLOVARJI.SI seznam slovenskih spletnih slovarjev.
16. Andersen MR et al. (2012) Kinect depth sensor evaluation for computer vision applications, (Department of Engineering, Aarhus University), Technical Report 6.
17. Balon A (1990) Vizualizacija, (Fakulteta za elektrotehniko in računalništvo, Univerza v Ljubljani), Diplomska naloga.
18. (year?) Biblatex – sophisticated bibliographies in latex.

Daljši slovenski povzetek

For Slovene students writing in English: V primeru pisanja v angleščini morate pripraviti razširjeni slovenski povzetek, ki naj bo dolg predvidoma 10 % dolžine celotnega zaključnega dela (brez dodatkov), torej pri dolžini 10 strani naj bo ta povzetek dolg 1 stran.

Appendices

A. Examples of Using Appendices. In this section, we provide some examples of using appendices. Here we place material that is useful for readers but too extensive to include in the main body of the final thesis, e.g., additional experiments or ablation studies, extra visual examples, etc. An example of additional results is shown in Table 3.

Table 3. A fictional example of evaluating the performance of various recognition architectures. Arrows in the table header indicate whether a higher (↑) or lower (↓) metric value denotes better performance. The best-performing model in each category is highlighted in bold and shaded. Avg. Eff. denotes average efficiency.

		Unsupervised Set (UERC)			Supervised Set			Avg. Eff.	
Architecture	**Params**	**Accuracy (%) ↑**	**AUC ↑**	**EER (%) ↓**	**Accuracy (%) ↑**	**AUC ↑**	**EER (%) ↓**	**Latency [ms] ↓**	**Size [M] ↓**
ResNet-50	25.6M	88.4	0.921	5.12	95.1	0.970	2.10	12.4	4.2
EfficientNet-B4	19.3M	90.1	0.945	4.05	96.8	0.982	1.45	18.2	3.8
Vision Transformer	86.6M	92.5	0.961	3.20	97.5	0.988	1.10	24.8	8.5
GAN-Detect	45.1M	91.0	0.950	3.88	96.0	0.975	1.80	16.5	6.1
Mega-Base	32.4M	94.2	0.978	2.15	98.2	0.991	0.85	**10.2** ↓	**3.5** ↓
Mega-Large	95.8M	**96.8** ↑	**0.989** ↑	**1.05** ↓	**99.1** ↑	**0.998** ↑	**0.42** ↓	22.1	14.2
ResNet-50	25.6M	88.4	0.921	5.12	95.1	0.970	2.10	12.4	4.2
EfficientNet-B4	19.3M	90.1	0.945	4.05	96.8	0.982	1.45	18.2	3.8
Vision Transformer	86.6M	92.5	0.961	3.20	97.5	0.988	1.10	24.8	8.5
GAN-Detect	45.1M	91.0	0.950	3.88	96.0	0.975	1.80	16.5	6.1